國立臺北科技大學
National Taipei University of Technology

資料結構

Data Structure

Lab 09

姓名：廖彥瑄

學號：10810030

Lab09-Ex1.
Delete Node in a Binary Search Tree
Code

```
 #include <iostream>
using namespace std;

// ========================
// 樹的節點類別
// ========================
class TreeNode {
public:
    int data;           // 節點儲存的資料
    TreeNode* left;     // 指向左子節點
    TreeNode* right;    // 指向右子節點

    // 節點建構子：設定資料並初始化左右指標為空
    TreeNode(int val) : data(val), left(nullptr), right(nullptr) {}
};

// ========================
// 二元搜尋樹類別
// ========================
class BinarySearchTree {
private:
    TreeNode* root; // 根節點指標

    // --- 私有的遞迴插入方法 ---
    TreeNode* insert(TreeNode* node, int val) {
        if (node == nullptr) {
            return new TreeNode(val); // 找到插入位置，建立新節點
        }

        if (val < node->data) {
            node->left = insert(node->left, val); // 插入左子樹
        }
        else if (val > node->data) {
            node->right = insert(node->right, val); // 插入右子樹
```

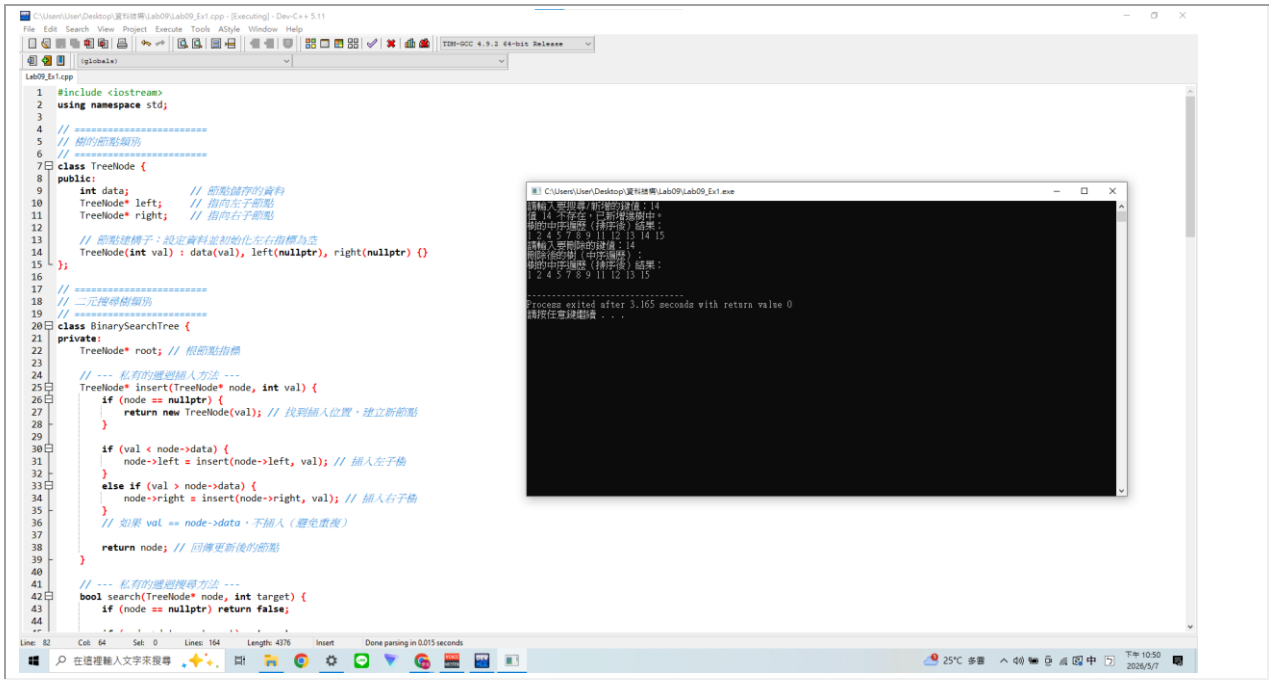
```
        }
        // 如果 val == node->data，不插入（避免重複）

        return node; // 回傳更新後的節點
    }

    // --- 私有的遞迴搜尋方法 ---
    bool search(TreeNode* node, int target) {
        if (node == nullptr) return false;

        if (node->data == target) return true;
        else if (target < node->data) return search(node->left, target); // 繼續在
左子樹找
        else return search(node->right, target);// 繼續在右子樹找
    }

    // --- 私有的中序遍歷方法（左-根-右）---
    void inOrderTraversal(TreeNode* node) {
        if (node == nullptr) return;

        inOrderTraversal(node->left);   // 先走訪左子樹
        cout << node->data << " ";   // 印出目前節點的資料
        inOrderTraversal(node->right); // 再走訪右子樹
    }

    // --- 私有的刪除方法 ---
    TreeNode* deleteNode(TreeNode* node, int val){
        if(node==nullptr)return nullptr;//沒有可以刪的東西
        if(val<node->data){
            node->left=deleteNode(node->left,val);//刪右子樹的
        }
        else if(val>node->data){
            node->right=deleteNode(node->right,val);//刪左子樹的
        }
        else{
            if(node->left==nullptr){//沒有左子樹
                TreeNode* temp=node->right;//右子樹存起來
                delete node;//刪掉自己
```

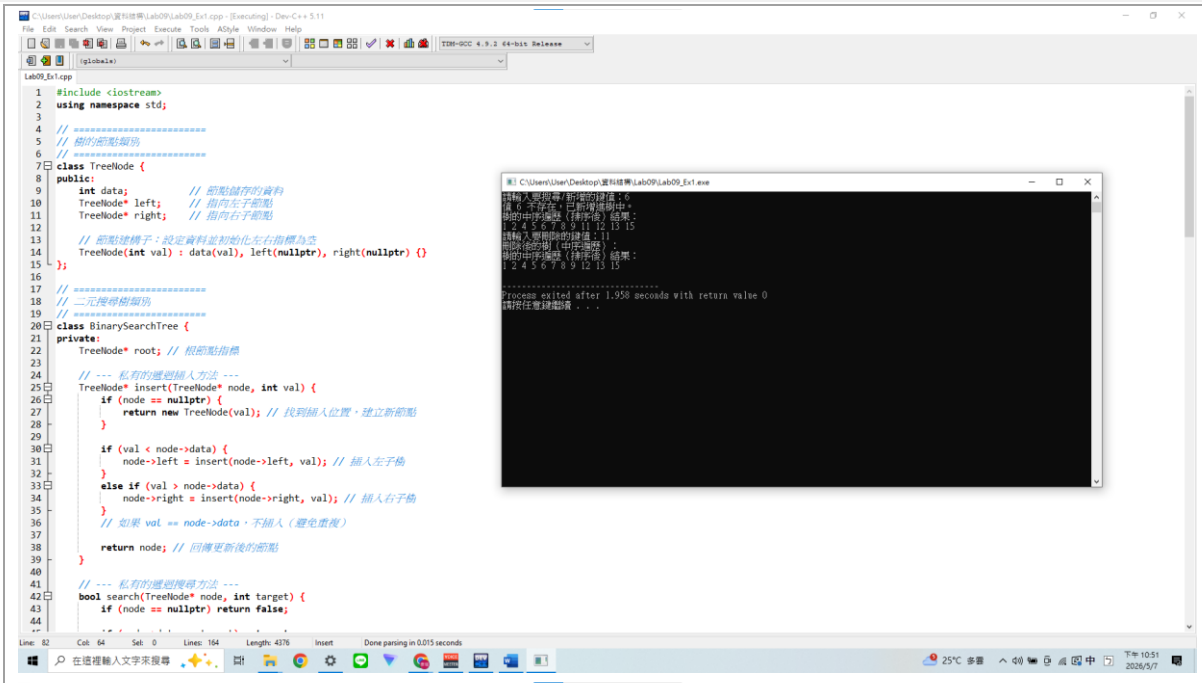
```
                return temp;//接上原本的右子樹
            }
            else if(node->right==nullptr){//沒有右子樹
                TreeNode* temp=node->left;//左子樹存起來
                delete node;
                return temp;
            }
            else{
                TreeNode* temp=findMin(node->right);//從右邊遞補
                node->data=temp->data;
                node->right=deleteNode(node->right,temp->data);
            }
        }
        return node;
    }

    // --- 私有的找最小值方法 ---
    TreeNode* findMin(TreeNode* node) {
        while (node->left != nullptr) {
            node = node->left;
        }
        return node;
    }

public:
    // --- 建構子：初始化為空樹 ---
    BinarySearchTree() : root(nullptr) {}

    // --- 公開插入方法 ---
    void insert(int val) {
        root = insert(root, val);
    }

    // --- 公開搜尋方法 ---
    bool search(int target) {
        return search(root, target);
    }
```

```
    // --- 公開中序遍歷方法 ---
    void displayInOrder() {
        cout << "樹的中序遍歷（排序後）結果：" << endl;
        inOrderTraversal(root);
        cout << endl;
    }

    // --- 公開刪除方法 ---
    void deleteValue(int val) {
        root = deleteNode(root, val);
    }
};

int main() {
    BinarySearchTree bst; // 建立一棵空的樹

    // 預設插入的資料
    int values[] = { 7, 1, 4, 2, 8, 13, 12, 11, 15, 9, 5 };
    int n = sizeof(values) / sizeof(values[0]);

    // 把資料插入樹中
    for (int i = 0; i < n; ++i) {
        bst.insert(values[i]);
    }

    // 讓使用者輸入要插入的鍵值
    int key;
    cout << "請輸入要搜尋/新增的鍵值：";
    cin >> key;

    if (bst.search(key)) {
        cout << "值 " << key << " 已存在樹中，不新增。" << endl;
    }
    else {
        cout << "值 " << key << " 不存在，已新增進樹中。" << endl;
        bst.insert(key);
    }
```

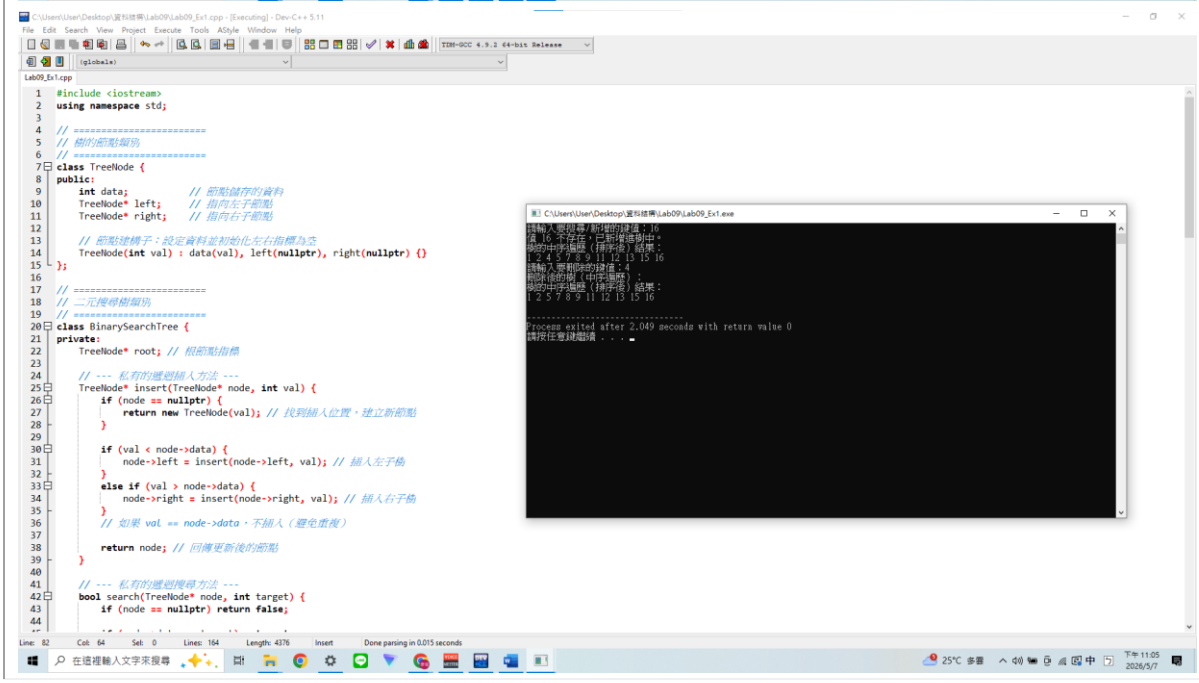
```
    // 顯示目前樹的中序遍歷結果
    bst.displayInOrder();

    // 讓使用者輸入要刪除的鍵值
    int deleteKey;
    cout << "請輸入要刪除的鍵值：";
    cin >> deleteKey;

    // 刪除節點
    bst.deleteValue(deleteKey);

    // 顯示刪除後的樹
    cout << "刪除後的樹（中序遍歷）：" << endl;
    bst.displayInOrder();

    return 0;
}
```

Result



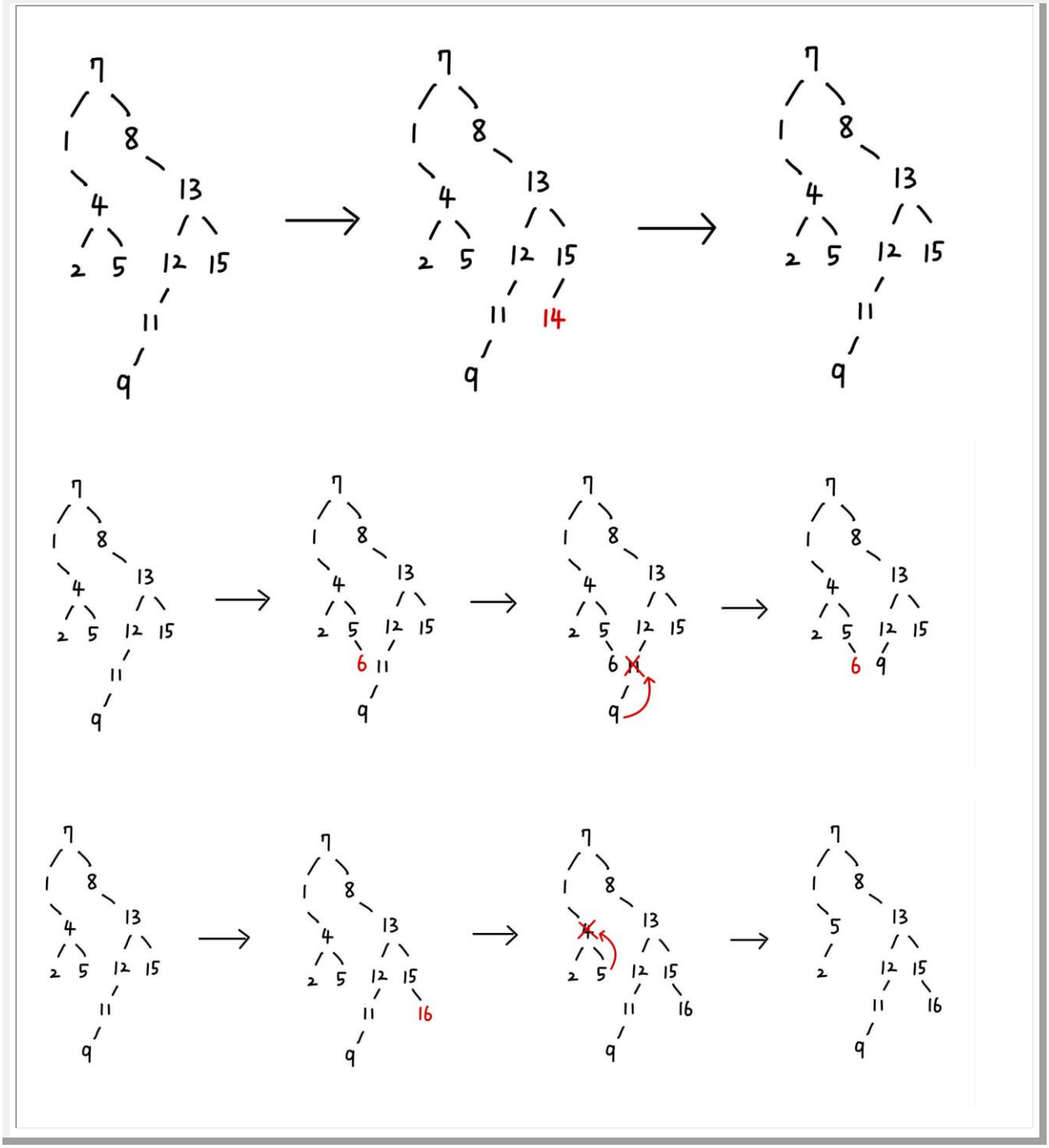

C:\Users\User\Desktop\資料結構\Lab09\Lab09_Ex1.cpp - [Executing] - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
TDM-GCC 4.9.2 64-bit Release
(globals)
Lab09_Ex1.cpp
#include <iostream>
using namespace std;
// ========================
// 樹的節點類別
// ========================
class TreeNode {
public:
int data; // 節點儲存的資料
TreeNode* left; // 指向左子節點
TreeNode* right; // 指向右子節點
// 節點建構子：設定資料並初始化左右指標為空
TreeNode(int val) : data(val), left(nullptr), right(nullptr) {}
};
// ========================
// 二元搜尋樹類別
// ========================
class BinarySearchTree {
private:
TreeNode* root; // 根節點指標
// --- 私有的遞迴插入方法 ---
TreeNode* insert(TreeNode* node, int val) {
if (node == nullptr) {
return new TreeNode(val); // 找到插入位置，建立新節點
}
if (val < node->data) {
node->left = insert(node->left, val); // 插入左子樹
}
else if (val > node->data) {
node->right = insert(node->right, val); // 插入右子樹
}
// 如果 val == node->data，不插入（避免重複）
return node; // 回傳更新後的節點
}
// --- 私有的遞迴搜尋方法 ---
bool search(TreeNode* node, int target) {
if (node == nullptr) return false;
C:\Users\User\Desktop\資料結構\Lab09\Lab09_Ex1.exe
請輸入要搜尋/新增的鍵值：6
值 6 不存在，已新增進樹中。
樹的中序遍歷（排序後）結果：
1 2 4 5 6 7 8 9 11 12 13 15
請輸入要刪除的鍵值：11
刪除後的樹（中序遍歷）：
樹的中序遍歷（排序後）結果：
1 2 4 5 6 7 8 9 12 13 15

Process exited after 1.958 seconds with return value 0
請按任意鍵繼續 . . .
Line: 82 Col: 64 Sel: 0 Lines: 164 Length: 4376 Insert Done parsing in 0.015 seconds
在這裡輸入文字來搜尋
25°C 多雲
下午 10:51
2026/5/7


C:\Users\User\Desktop\資料結構\Lab09\Lab09_Ex1.cpp - [Executing] - Dev-C++ 5.11
File Edit Search View Project Execute Tools AStyle Window Help
TDM-GCC 4.9.2 64-bit Release
(globals)
Lab09_Ex1.cpp
#include <iostream>
using namespace std;
// ========================
// 樹的節點類別
// ========================
class TreeNode {
public:
int data; // 節點儲存的資料
TreeNode* left; // 指向左子節點
TreeNode* right; // 指向右子節點
// 節點建構子：設定資料並初始化左右指標為空
TreeNode(int val) : data(val), left(nullptr), right(nullptr) {}
};
// ========================
// 二元搜尋樹類別
// ========================
class BinarySearchTree {
private:
TreeNode* root; // 根節點指標
// --- 私有的遞迴插入方法 ---
TreeNode* insert(TreeNode* node, int val) {
if (node == nullptr) {
return new TreeNode(val); // 找到插入位置，建立新節點
}
if (val < node->data) {
node->left = insert(node->left, val); // 插入左子樹
}
else if (val > node->data) {
node->right = insert(node->right, val); // 插入右子樹
}
// 如果 val == node->data，不插入（避免重複）
return node; // 回傳更新後的節點
}
// --- 私有的遞迴搜尋方法 ---
bool search(TreeNode* node, int target) {
if (node == nullptr) return false;
C:\Users\User\Desktop\資料結構\Lab09\Lab09_Ex1.exe
請輸入要搜尋/新增的鍵值：16
值 16 不存在，已新增進樹中。
樹的中序遍歷（排序後）結果：
1 2 4 5 6 7 8 9 11 12 13 15 16
請輸入要刪除的鍵值：4
刪除後的樹（中序遍歷）：
樹的中序遍歷（排序後）結果：
1 2 5 7 8 9 11 12 13 15 16

Process exited after 2.049 seconds with return value 0
請按任意鍵繼續 . . .
Line: 82 Col: 64 Sel: 0 Lines: 164 Length: 4376 Insert Done parsing in 0.015 seconds
在這裡輸入文字來搜尋
25°C 多雲
下午 11:05
2026/5/7

Discussion


7
1 8
4 13
2 5 12 15
11
9
14
6
16